

0.1 Cơ chế tích lũy lãi Lazy Accrual + Global Interest Index

Cơ chế tích lũy lãi suất là thành phần quan trọng bậc nhất của giao thức vay và cho vay phi tập trung. Mục ?? đã mô tả chi tiết cơ chế hoạt động của Lazy Accrual kết hợp Global Interest Index, bao gồm công thức tính `accrueInterest` và cách cập nhật vị thế người dùng. Cách tiếp cận chỉ số lãi toàn cục được tham chiếu từ các giao thức lending phổ biến như Compound v2 [1]. Mục này tập trung phân tích lý do lựa chọn cơ chế Lazy Accrual thay vì các giải pháp thay thế, cùng với các tính chất mà Global Interest Index đảm bảo. Bài toán cốt lõi là mỗi lần cập nhật trạng thái on-chain đều tốn chi phí gas (đặc biệt SSTORE tốn xấp xỉ 20.000 gas — xem mục ??), vậy làm thế nào để tích lũy lãi cho n người dùng mà vẫn tối ưu chi phí.

Đây chính là lý do hệ thống lựa chọn giải pháp Lazy Accrual thay cho Eager Accrual. Với cách tiếp cận Eager Accrual, mỗi khi có một giao dịch làm thay đổi trạng thái của market, giao thức phải cập nhật lại số dư đã tích lũy lãi của toàn bộ người dùng đang tham gia market đó, chứ không chỉ của người vừa tương tác. Điều này có nghĩa là chi phí xử lý của một lần thay đổi lãi suất sẽ tăng theo số lượng người dùng đang hoạt động trong market. Nếu ký hiệu U là số người dùng và T là số lần market phát sinh thay đổi cần cập nhật lãi suất trong một giai đoạn, thì tổng khối lượng cập nhật trạng thái sẽ tỷ lệ với $U \times T$. Nói cách khác, càng nhiều người dùng tham gia và càng nhiều giao dịch xảy ra, lượng storage write mà giao thức phải thực hiện càng tăng nhanh. Trong môi trường blockchain, nơi mỗi lần ghi storage đều tốn gas đáng kể, cách tiếp cận này không còn phù hợp ở quy mô lớn. Vì vậy, giao thức sử dụng Lazy Accrual: chỉ khi một người dùng trực tiếp tương tác, hệ thống mới tính và cập nhật phần lãi liên quan đến người đó, thay vì cập nhật đồng loạt cho toàn bộ market.

Tuy đã giải quyết được bài toán về chi phí cho hệ thống, nhưng Lazy Accrual lại mang đến một vấn đề khó khăn hơn khi nghiệp vụ hệ thống yêu cầu 2 ràng buộc liên quan đến lãi suất. Ràng buộc thứ nhất là không kỳ hạn: người dùng có thể gửi hoặc rút tại bất kỳ block nào, không thể biết trước thời điểm tương tác. Ràng buộc thứ hai là lãi suất thả nổi: lãi suất thay đổi mỗi khi tỷ lệ sử dụng vốn thay đổi, tức là thay đổi mỗi khi có bất kỳ giao dịch nào của bất kỳ người dùng nào xảy ra trên cùng một tài sản. Hệ quả là trong khoảng thời gian giữa 2 lần tương tác của người dùng A, lãi suất có thể đã thay đổi hàng chục lần do các giao dịch của người dùng B, C, D. Vậy câu hỏi đặt ra là: làm thế nào để đảm bảo tính công bằng, tính đúng số tiền lãi mà người dùng A được nhận mà không cần lưu lịch sử toàn bộ các lần thay đổi lãi suất, đồng thời tối ưu chi phí gas mà người dùng phải chịu mỗi lần thực hiện Lazy Accrual.

Đối với bài toán này có thể nghĩ đến 3 cách tiếp cận như sau: Cách tiếp cận thứ nhất là cập nhật số dư của tất cả người dùng mỗi khi lãi suất thay đổi. Cách này cho kết quả chính xác nhưng chi phí $O(n)$ storage write mỗi giao dịch, với n người dùng, mỗi giao dịch deposit của bất kỳ ai cũng phải cập nhật số dư của tất cả người còn lại, hoàn toàn không khả thi trên blockchain nơi mỗi storage write tốn gas. Cách tiếp cận thứ hai là lưu toàn bộ lịch sử thay đổi lãi suất theo thời gian và tính tổng tích phân khi người dùng rút tiền. Cách này tránh được $O(n)$ write nhưng chi phí đọc $O(k)$ với k là số lần thay đổi lãi suất có thể lên đến hàng nghìn lần trong vài ngày, và gas chi phí đọc lịch sử tăng tuyến tính theo thời gian người dùng gửi tiền. Cách tiếp cận thứ ba, cũng là giải pháp được giao thức lựa chọn là Global Interest Index: thay vì lưu lịch sử, giao thức duy trì một chỉ số tích lũy tăng dần liên tục. Mỗi khi lãi suất thay đổi, chỉ số này được cập nhật một lần duy nhất ($O(1)$). Người dùng chỉ cần lưu snapshot của chỉ số tại thời điểm gửi tiền; số lãi tích lũy được tính bằng tỷ lệ giữa chỉ số hiện tại và snapshot, bất kể lãi suất đã thay đổi bao nhiêu lần ở giữa.

Thiết kế này đảm bảo được 3 tính chất đồng thời: (i) chính xác: người dùng nhận đúng lãi tương ứng với thời gian gửi thực tế tính đến giây, bất kể lãi suất thả nổi biến động như thế nào; (ii) công bằng: không có sự khác biệt giữa người gửi sớm và muộn, mỗi người nhận lãi đúng theo phần vốn và thời gian của mình; (iii) tiết kiệm gas: chi phí tích lũy lãi không phụ thuộc vào số người dùng trong hệ thống, mỗi giao dịch chỉ cập nhật 2 giá trị index toàn cục thay vì n số dư người dùng, từ đó giảm tải đáng kể chi phí gas mà người dùng phải chi trả.

0.2 Kiến trúc bảo mật 5 tầng của LendingPool

Mục ?? đã mô tả chi tiết kiến trúc bảo mật 5 tầng của LendingPool. Mục này tập trung phân tích lý do thiết kế và ba quyết định phi hiển nhiên.

Một cách tiếp cận phổ biến khi bảo mật smart contract là tập trung toàn bộ kiểm tra vào một modifier tổng hợp hoặc một hàm validate duy nhất. Cách này đơn giản khi cài đặt nhưng tạo ra single point of failure: nếu logic validate có lỗi hoặc bị bypass, toàn bộ hệ thống mất bảo vệ. LendingPool áp dụng nguyên tắc defense in depth - phòng vệ theo chiều sâu bằng cách phân chia bảo mật thành 5 tầng hoàn toàn độc lập. Mỗi tầng phòng vệ một loại rủi ro cụ thể, và sự thất bại của một tầng không làm suy yếu các tầng còn lại. Kẻ tấn công phải vượt qua đồng thời tất cả 5 tầng để có thể gây hại cho giao thức.

Quyết định thiết kế phi hiển nhiên thứ nhất là sử dụng onlyLiquidation thay vì onlyController. Hai hook thanh lý seizeCollateral() và repayFromLiquidation() được bảo vệ bởi modifier onlyLiquidation thay vì onlyController như toàn bộ các

hàm admin khác. Lý do là trong runtime, ProtocolController không phải người gọi các hàm này, Liquidation contract mới là người gọi trực tiếp trong quá trình thanh lý. Nếu dùng onlyController, ProtocolController sẽ phải làm trung gian cho mọi lần thanh lý, tăng chi phí gas không cần thiết và vi phạm nguyên tắc separation of concerns: ProtocolController chỉ nên là entry point cho hành động quản trị, không phải cho hành động nghiệp vụ thời gian thực. Hệ quả của thiết kế này là LendingPool biết chính xác ai được phép gọi từng hàm: admin đi qua Controller, liquidation đi qua Liquidation contract, hai luồng hoàn toàn tách biệt, không thể nhầm lẫn hay lạm dụng.

Quyết định thiết kế thứ hai nằm ở việc kiểm tra trùng lặp có chủ ý: Trong tầng logic nghiệp vụ, cả LendingPool lẫn Liquidation contract đều kiểm tra $\text{seizeAmount} \leq \text{currentBorrowerDeposit}$, dù Liquidation đã cap giá trị này trước khi gọi hook. Tương tự, cả hai đều kiểm tra $\text{repayAmount} \leq \text{currentBorrow}$. Đây không phải lỗi thiết kế hay code thừa, đây là defense in depth có chủ ý ở tầng business logic. Lý do là nếu trong tương lai Liquidation contract được thay thế bằng một implementation mới có lỗi logic hoặc bị upgrade sai, LendingPool vẫn có lớp bảo vệ độc lập. Hai smart contract phòng vệ độc lập nhau đảm bảo ngay cả khi một bên bị compromise, bên còn lại vẫn giữ được tính toàn vẹn của trạng thái.

Quyết định thiết kế phi hiển nhiên cuối cùng nằm ở tầng khởi tạo contract khi sử dụng “_disableInitializers()” trong constructor. Với UUPS proxy pattern, tồn tại một attack vector ít được biết đến: kẻ tấn công gọi initialize() trực tiếp trên implementation contract (không phải proxy), chiếm quyền controller của implementation, sau đó dùng quyền đó để upgrade proxy sang một implementation độc hại. Lỗ hổng này không hiển nhiên vì trong điều kiện bình thường, người dùng chỉ tương tác qua Proxy, implementation trông có vẻ vô hại. Tuy nhiên vì implementation cũng là một smart contract on-chain với địa chỉ công khai, bất kỳ ai cũng có thể gọi hàm của nó. Giải pháp là gọi _disableInitializers() trong constructor của implementation. Lệnh này khóa cứng mọi khả năng khởi tạo trên implementation contract ngay khi deploy [2].

0.3 Kiến trúc backend đa tiến trình event-driven

Mục ?? đã mô tả kiến trúc đa tiến trình event-driven với 5 worker độc lập (blc-indexer, blc-worker, croner, noti-worker, http-server) và cách chúng phối hợp qua RabbitMQ và Redis (Hình 4.10). Mục này phân tích lý do kiến trúc tại sao không sử dụng monolith thuần túy.

Xây dựng backend dưới dạng một ứng dụng monolith duy nhất là cách tiếp cận đơn giản với một process vừa quét blockchain, vừa xử lý event, vừa chạy cron job,

vừa phục vụ HTTP request. Tuy nhiên, cách này tạo ra 3 vấn đề nghiêm trọng. Thứ nhất là tính sẵn sàng: nếu blockchain node RPC bị chậm khiến indexer block, toàn bộ HTTP server cũng bị ảnh hưởng, người dùng không truy cập được giao diện chỉ vì một vấn đề với RPC provider. Thứ hai là khả năng mở rộng: khi lượng event tăng cao, không thể scale riêng phần xử lý event mà không scale cả HTTP server đang chạy tốt. Kiến trúc nhiều worker này tăng tối đa khả năng mở rộng theo chiều dọc của hệ thống, nếu một worker là không đủ chịu tải, trong khi server còn nhiều tài nguyên dư thừa, ta có thể dễ dàng thêm nhiều instance của worker đó, từ đó tăng khả năng xử lý của một nhiệm vụ. Thứ ba là độ phức tạp: code xử lý blockchain, code nghiệp vụ database, và code phục vụ HTTP request trộn lẫn nhau, khó bảo trì và dễ gây side effect không lường trước.

Tại sao lại cần message queue (RabbitMQ) thay vì xử lý trực tiếp? Khi indexer phát hiện một event từ blockchain, cách đơn giản nhất là gọi thẳng hàm xử lý trong cùng process. Cách này đồng bộ và dễ debug, nhưng tạo ra vấn đề mất dữ liệu: nếu worker crash ngay sau khi indexer đã xử lý event nhưng trước khi lưu vào database, event đó biến mất hoàn toàn. RabbitMQ giải quyết vấn đề này thông qua cơ chế acknowledgment: message chỉ bị xóa khỏi queue sau khi worker gửi ack xác nhận đã xử lý thành công. Nếu worker crash trước khi ack, RabbitMQ tự động requeue message, event sẽ được xử lý lại khi worker khởi động trở lại [3]. Đây là đảm bảo at-least-once delivery, phù hợp với dữ liệu tài chính nơi mất event là không chấp nhận được. Hệ quả trực tiếp là hệ thống có khả năng self-healing: bất kỳ worker nào crash và restart đều tự động tiếp tục từ điểm bị gián đoạn mà không cần can thiệp thủ công.

At-least-once delivery dẫn đến khả năng xử lý trùng lặp, cơ chế idempotency để xử lý hệ quả này đã được trình bày về mặt lý thuyết tại mục ??, chi tiết cài đặt với unique constraint tại mục ??.

0.4 Thiết kế blockchain indexer: Polling thay vì Subscribe Event

Bài toán khó của việc index blockchain để lưu trữ dữ liệu chính là việc blockchain indexer cần liên tục theo dõi các event phát sinh từ smart contract và vào database. Câu hỏi thiết kế cốt lõi là: dùng cơ chế nào để nhận event từ blockchain.

Đây là quyết định có ảnh hưởng lớn đến tính tin cậy của toàn bộ hệ thống backend, vì một event bị bỏ sót đồng nghĩa với dữ liệu trong database lệch khỏi trạng thái thực tế on-chain, sai lệch không thể tự phát hiện và sửa chữa nếu không có cơ chế phục hồi. Ban đầu có hai giải pháp có thể được nghĩ tới, một là thiết kế indexer theo hướng Subscribe Event (eth_subscribe / WebSocket Listener) và hai là theo hướng Polling theo khoảng block với queryFilter. Hai giải pháp này được so sánh

ngắn gọn trong bảng 0.1.

Đối với giải pháp đầu tiên, thực hiện kết nối WebSocket đến RPC node và đăng ký lắng nghe event theo thời gian thực thông qua `eth_subscribe`. Khi smart contract phát ra event, RPC node đẩy ngay về client mà không cần client hỏi. Ưu điểm của giải pháp này chính là độ trễ cực thấp, gần như tức thì, event được nhận gần như ngay lập tức sau khi block được mint. Nhưng với ưu điểm không có nhiều hấp dẫn, nhược điểm của nó lại cực kỳ nghiêm trọng: Đầu tiên là mất event khi kết nối đứt do kết nối WebSocket không ổn định, bị ngắt do mạng, timeout, worker bị chết hoặc RPC node restart. Trong khoảng thời gian kết nối bị đứt và kết nối lại, toàn bộ event phát sinh đều bị mất vĩnh viễn. Thư viện `ethers.js` có cơ chế reconnect tự động nhưng không lưu lại event đã bỏ lỡ trong khoảng downtime [4]. Thứ hai, khi đã mất event rồi, thì lại không có cơ chế hồi phục bởi consumer không biết mình đã nhận event đến block nào. Khi worker khởi động lại sau sự cố, không có cách nào xác định điểm tiếp tục, chỉ có thể bắt đầu lắng nghe từ block hiện tại, để lại khoảng trống dữ liệu không thể khôi phục.

Vì vậy, giải pháp thể hiện ưu điểm tối ưu hơn hoàn toàn, và được lựa chọn chính là giải pháp còn lại: sử dụng croner chạy định kỳ quét blockchain. Blockchain indexer sẽ định kỳ gọi `contract.queryFilter("*", fromBlock, toBlock)` để lấy toàn bộ event trong một khoảng block xác định. Trạng thái tiến độ được lưu bền vững vào database qua `lastScannedBlock`. Nhược điểm dễ nhận thấy so với giải pháp đầu đó là Latency cao hơn, event được phát hiện muộn hơn tối đa một chu kỳ cron (12 giây với Ethereum). Tuy nhiên, với ứng dụng lending protocol, độ trễ này hoàn toàn chấp nhận được: các hành động như gửi, vay, thanh lý không đòi hỏi phản ứng trong millisecond. Trong khi đó, thiết kế polling cho phép hệ thống phục hồi hoàn toàn sau downtime: Khi worker khởi động lại sau sự cố bất kể bao lâu, nó đọc `lastScannedBlock` từ database và tiếp tục chính xác từ block tiếp theo. Không có event nào bị bỏ sót, mọi block trong khoảng thời gian downtime đều được quét đầy đủ khi worker hoạt động trở lại. Bên cạnh đó, nó còn cho phép kiểm soát lưu lượng RPC với `MAX_BLOCK_RANGE` giới hạn số block mỗi lần quét, tránh vi phạm rate limit khi worker cần bắt kịp sau downtime dài đồng thời dễ debug và kiểm tra khi mà trạng thái tiến độ (`lastScannedBlock`) hiển thị rõ ràng trong database, tại bất kỳ thời điểm nào đều biết chính xác indexer đã xử lý đến block nào và đang xử lý block nào.

Tiêu chí	Subscribe Event	Polling + queryFilter
Latency nhận event	Rất thấp (< 1s)	Thấp ($\leq$ block time)
Phục hồi sau downtime	Không — mất event trong khoảng downtime	Hoàn toàn — tiếp tục từ lastScannedBlock
Ổn định kết nối	Phụ thuộc WebSocket — dễ mất kết nối	HTTP request độc lập — không có kết nối dài hạn
Kiểm soát lưu lượng RPC	Khó — event đến bất kỳ lúc nào	Dễ — MAX_BLOCK_RANGE giới hạn mỗi lần quét
Khả năng debug	Khó — luồng event bất đồng bộ	Dễ — trạng thái lưu rõ ràng trong database

Bảng 0.1: So sánh hai hướng tiếp cận index blockchain

0.5 Kiến trúc quản trị 2 tầng: Multisig và Timelock

Mục ?? đã trình bày thiết kế kỹ thuật của ProtocolController (Single Entry Point, Atomic Multi-Contract Update) và ProtocolTimelock (kế thừa OpenZepplin TimelockController). Mục này phân tích lý do kiến trúc tại sao cần Multisig và Timelock, thay vì mô hình single admin.

Mô hình quản trị đơn giản nhất là một địa chỉ admin duy nhất có toàn quyền thay đổi giao thức, tuy nhiên mô hình này lại dính phải điểm yếu chí mạng: single point of failure. Mô hình này có 3 rủi ro nghiêm trọng: Thứ nhất là rủi ro mất private key: nếu key của admin bị mất, không ai có thể quản lý giao thức, không thể thêm tài sản mới, không thể xử lý khẩn cấp, không thể upgrade khi phát hiện lỗ hổng. Thứ hai là rủi ro bị compromise: nếu key bị lộ hoặc bị tấn công, kẻ tấn công có thể thực hiện bất kỳ hành động quản trị nào ngay lập tức, không có cơ chế nào để chặn lại. Thứ ba là thiếu minh bạch: cộng đồng không có cách nào biết trước các thay đổi sắp xảy ra, không có thời gian để phát hiện và phản ứng với các thay đổi bất thường.

Để giải quyết các rủi ro này, hệ thống quyết định triển khai sử dụng ví đa chữ ký đi kèm với timelock. Vậy làm thế nào multisig có thể giải quyết được rủi ro thứ nhất và rủi ro thứ hai. Safe Multisig yêu cầu chữ ký từ k trong n người nắm giữ key trước khi một hành động được thực thi [5]. Thiết kế này loại bỏ single point of failure theo cả hai chiều: mất một key không làm mất quyền quản trị (vì vẫn còn $k-1$ key khác), và lộ một key không đủ để kẻ tấn công thực hiện hành động (vì cần đủ k chữ ký). Điều quan trọng cần nhấn mạnh là Multisig không phải là tính năng tiện lợi, đây là yêu cầu bảo mật tối thiểu cho bất kỳ DeFi protocol nào quản lý tài sản của người dùng. Thiếu Multisig đồng nghĩa với việc toàn bộ tài sản trong giao thức phụ thuộc vào tính an toàn của một private key duy nhất.

Timelock đóng vai trò trong việc xử lý rủi ro còn lại trong quản trị. Nó giải quyết vấn đề minh bạch bằng cách áp đặt độ trễ bắt buộc giữa lúc đề xuất và lúc thực thi [2]. Trong khoảng thời gian này, toàn bộ hành động admin đều public trên blockchain, bất kỳ ai cũng có thể đọc và kiểm tra. Cộng đồng có đủ thời gian để phát hiện thay đổi bất thường và phản ứng: rút tài sản, đề xuất cancel, hoặc cảnh báo công khai. Ngoài vai trò minh bạch, Timelock còn là lớp phòng vệ thứ hai cho trường hợp Multisig bị compromise: kẻ tấn công có đủ k chữ ký vẫn phải chờ minDelay trước khi hành động có hiệu lực, tạo ra cửa sổ thời gian để phát hiện và phản ứng.

Một hệ quả quan trọng của kiến trúc này là Atomic Multi-Contract Update. Vì tất cả hành động admin đều đi qua một entry point duy nhất là ProtocolController, một hành động có thể cập nhật nhiều smart contract trong cùng một transaction on-chain. Ví dụ khi thêm tài sản mới, supportMarketWithChainlinkFeed() gọi đồng thời LendingPool.supportMarket() và PriceRouter.setChainlinkFeed(), đảm bảo không bao giờ có trạng thái trung gian: tài sản được thêm vào LendingPool nhưng chưa có nguồn giá trong PriceRouter, hoặc ngược lại. Nếu một trong hai lệnh fail, cả transaction bị revert, đây là tính atomicity của blockchain được tận dụng triệt để vào logic quản trị.

Đối với thư viện timelock on-chain, OpenZeppelin đã cung cấp sẵn TimelockController có thể dùng trực tiếp mà không cần deploy smart contract riêng. Việc tạo ProtocolTimelock kế thừa lại mà không thêm logic mới nhìn có vẻ thừa. Lý do thực tế có 2 mặt. Về mặt kỹ thuật, smart contract riêng có địa chỉ on-chain riêng, frontend, backend và các công cụ monitoring có thể theo dõi đúng smart contract này mà không nhầm với các TimelockController khác có thể tồn tại trên cùng mạng. Về mặt cộng đồng, khi smart contract được verify trên Etherscan với tên ProtocolTimelock, người dùng nhìn vào transaction history ngay lập tức hiểu đây là timelock của giao thức, tăng niềm tin và tính minh bạch.